

CS 165 Lab 1 Find Trigger Fix

Student Instructions

In this lab, you will investigate a small C record manager that contains five seeded memory safety bugs. You will create a trigger for each bug, explain the failure, make a focused fix, and verify that normal behavior still works.

Generate your program with the student identifier (SID). The generator uses that identifier to create your individual source file, so use the same identifier throughout the lab and when you submit your work.

The Program

The generated program is named `recman`. It reads a script of record-management commands. The supplied sample scripts use ordinary inputs and should run normally. Your own scripts will exercise the hidden bugs.

Command	Meaning
<code>add <id></code>	Create an empty record.
<code>name <id> <text></code>	Set a record's fixed-size name field.
<code>str <id> <text></code>	Set a record's heap string.
<code>app <id> <text></code>	Append text to a record's heap string.
<code>grow <id> <n></code>	Reserve an n-entry integer array.
<code>link <src> <dst></code>	Make <code>dst</code> refer to <code>src</code> .
<code>del <id></code>	Delete a record.
<code>show <id></code>	Print a record.

Commands and Their Meaning

Command	Purpose
<code>python3 make_recman.py <student-id></code>	Generate recman.c, a Makefile, and the tests directory for your personal instance.
<code>mkdir -p triggers</code>	Create the folder that will hold your five trigger scripts.
<code>make</code>	Build recman with AddressSanitizer. Use this build to find and verify the bugs.
<code>./recman tests/sample1.txt</code>	Run recman with a supplied benign script.
<code>./recman triggers/bug1.txt</code>	Run one of your own trigger scripts.
<code>make tar</code>	Create the submission archive named lab1-submission.tgz.

Additional Development Commands

- `make clean` removes the current recman executable before you rebuild.
- `make plain` builds recman without AddressSanitizer and replaces the current executable. Run `make` again to restore the ASan build.
- `valgrind ./recman <script>` checks a non-ASan build for heap-related invalid reads, writes, and use-after-free errors when Valgrind is available.
- `gdb --args ./recman <script>` starts recman in GDB so you can pause, inspect variables, and view a backtrace.

How to Run the Lab

Work from inside the `student_kit` folder. Replace `<student-id>` with your SID.

```
cd student_kit
python3 make_recman.py <student-id>
mkdir -p triggers
touch writeup.md
make
./recman tests/sample1.txt
```

The program should build and the sample should run without an AddressSanitizer report. Run `tests/sample2.txt` and `tests/sample3.txt` as additional normal-behavior checks whenever you change `recman.c`.

Bugs to Find

Your generated `recman.c` contains one bug from each category below. The source locations and values differ by student. Find the bugs from the code and your own command scripts rather than from a copied trigger.

Bug	Category	CWE
Bug 1	Stack buffer overflow	CWE 121
Bug 2	Heap buffer overflow	CWE 122
Bug 3	Use after free	CWE 416
Bug 4	Integer overflow that leads to a heap overflow	CWE 190 then CWE 122
Bug 5	NULL pointer dereference	CWE 476

What You Need to Do

Complete the following work for each of the five bugs.

- Read `recman.c` and identify the unsafe operation and its CWE category.
- Create a command script that makes the original ASan build fail. Save it as `triggers/bug1.txt` through `triggers/bug5.txt`.
- Before you patch the code, preserve the first two lines of the AddressSanitizer report for your writeup.
- In `writeup.md`, explain the CWE, the relevant function or line, why your script triggers the bug, and the build-specific values you used. Use two to four sentences per bug.
- Make a minimal change to `recman.c` that fixes the bug without breaking normal behavior.

After all five fixes, rebuild with `make`. Run every trigger and all three sample scripts again. The patched ASan build should not abort for any of your triggers, and the samples should continue to work.

What to Submit

Your submission archive must contain the following items.

- `recman.c` with all five fixes
- `triggers/bug1.txt` through `triggers/bug5.txt`
- `writeup.md` with one entry for each bug

Create the archive from the generated lab directory with the following command.

```
make tar
```

Submit the resulting `lab1-submission.tgz` through the course submission system. Do not include compiled executables or a copied student kit in the archive.